Régulation adaptative d'un carrefour par Deep Q-Learning

Entraînement, évaluation contre baselines et analyse critique

Carrefour réel de Thiès (Sénégal) sous Eclipse SUMO

Mamadou Ahmadou Ka
Étudiant en Licence 1 Informatique
Département Informatique
Université Iba Der Thiam de Thiès (UIDT), Sénégal

8 Août 2026

Résumé

Ce document présente l'entraînement et l'évaluation d'un agent de Deep Q-Learning régulant un carrefour réel de Thiès (Sénégal) sous le simulateur microscopique Eclipse SUMO. L'agent a été entraîné sur 150 épisodes (108 000 décisions), la demande de trafic étant régénérée à chaque épisode avec une graine différente afin de contraindre la généralisation.

Le résultat central n'est pas la progression de l'agent par rapport à son initialisation aléatoire — qui ne prouve rien — mais sa comparaison à deux contrôleurs de référence sur cinq graines de trafic jamais vues à l'entraînement. Face au feu à programme fixe actuellement en place, le régulateur appris réduit le temps perdu par véhicule de 50,7 % (31,5 s → 15,5 s) et le temps d'attente de 62,2 % (23,8 s → 9,0 s). Un résultat contre-intuitif est également rapporté et expliqué : la baseline actionnée par détection de véhicules se révèle *moins* performante que le feu fixe (+8,7 % de temps perdu) sur ce carrefour à demande asymétrique.

Le rapport documente enfin les limites méthodologiques de l'expérimentation, dont un défaut de sélection du modèle détecté et corrigé en cours d'étude.

Table des matières

1 Introduction et Contexte Expérimental **3**
- 1.1 Formulation du problème de décision markovien . . . 3
- 1.2 Contrôle effectif du feu . . . 3

2 Protocole expérimental **4**

3 Résultats d'entraînement **4**
- 3.1 Progression de l'attente moyenne . . . 5
- 3.2 Décroissance de la perte . . . 5
- 3.3 Distribution des attentes moyennes . . . 6

4 Évaluation contre baselines **6**
- 4.1 Pourquoi la progression interne ne prouve rien . . . 6
- 4.2 Résultats . . . 6
- 4.3 Le gain moyen cache-t-il des véhicules sacrifiés ? . . . 7
- 4.4 Un résultat contre-intuitif : la baseline actionnée est moins bonne que le feu fixe 7

5 Analyse critique et limites **7**
- 5.1 Un défaut de sélection du modèle, détecté et corrigé . . . 7
- 5.2 L'agent a-t-il simplement ré-appris un cycle fixe ? . . . 8
- 5.3 Une phase abandonnée : optimisation réelle, et son coût . . . 9
- 5.4 Métrique de vitesse trompeuse sur les voies . . . 9
- 5.5 Équité par mouvement non garantie . . . 9
- 5.6 Portée limitée à un carrefour isolé . . . 10
- 5.7 Absence de répétitions indépendantes . . . 10
- 5.8 Écart entre simulation et terrain . . . 10

6 Conclusion et perspectives **10**
- 6.1 Rappels théoriques . . . 10
- 6.2 Résultats établis . . . 10
- 6.3 Travaux futurs . . . 11
- 6.4 Reproductibilité . . . 11

1 Introduction et Contexte Expérimental

La régulation de trafic par apprentissage par renforcement repose sur l'interaction entre un agent décisionnel et un environnement de simulation microscopique. L'étude porte sur le carrefour `cluster_6021368912_6083287698_6083287699_6083287700` de la ville de Thiès, comportant quatre phases vertes et huit voies entrantes contrôlées.

1.1 Formulation du problème de décision markovien

État. À chaque décision, l'agent observe un vecteur $\mathbf{s}_t \in \mathbb{R}^{13}$ composé de :
- 4 longueurs de file d'attente, agrégées par branche et normalisées ;
- 4 densités d'occupation, rapportées à la capacité de chaque groupe de voies ;
- l'encodage *one-hot* de la phase verte courante (4 composantes) ;
- 1 indicateur binaire signalant que la durée de vert minimale est écoulée.

Action. L'action $a_t \in \{0, 1, 2, 3\}$ désigne **l'indice de la prochaine phase verte**. Ce choix est important : un espace binaire $\{\text{STAY}, \text{SWITCH}\}$ ne permettrait à l'agent que de suivre ou d'accélérer un cycle prédéfini, sans jamais réordonner les phases. Les actions incompatibles avec les contraintes de sécurité (vert minimal de 10 s, vert maximal de 60 s) sont masquées : leur valeur Q est forcée à $-\infty$ avant la sélection.

Récompense. $r_t = (W_{t-1} - W_t)/100$, où W désigne le temps d'attente cumulé de tous les véhicules sur les voies contrôlées. La récompense est positive lorsque l'agent résorbe l'attente. Le coût d'un changement de phase est capturé naturellement par le temps de jaune, sans pénalité arbitraire.

Invariant temporel. Le jaune (3 s) est *inclus* dans l'intervalle de décision (5 s). Rester sur la phase courante et en changer consomment donc exactement la même durée simulée, ce qui rend les récompenses des deux branches comparables. Sans cet invariant, changer de phase coûterait mécaniquement plus de temps simulé et l'agent apprendrait à ne jamais commuter — un biais qui a effectivement été observé dans une version antérieure de l'implémentation.

1.2 Contrôle effectif du feu

L'agent commande le carrefour par `setRedYellowGreenState()`, et non par `setPhase()`. Cette distinction n'est pas cosmétique : `setPhase()` positionne la phase mais réarme sa durée programmée, si bien que SUMO poursuit ensuite le cycle statique de lui-même. L'agent croit alors piloter un feu qui, en réalité, continue de tourner seul.

Un test dédié (`rl.smoke_test`) vérifie cette propriété : le feu est laissé 60 s sans aucun ordre et doit conserver son état, là où le programme statique aurait basculé après 27 s. Un garde-fou (`assert_in_sync`) compare en outre l'état réel du feu à l'état interne de l'agent.

2 Protocole expérimental

TABLE 1 – Paramètres de l'expérimentation

Élément	**Valeur**
Algorithme	Double DQN, perte de Huber, cible en *soft update* ($\tau = 0{,}005$)
Architecture	Perceptron multicouche $13 \to 64 \to 64 \to 4$
Optimiseur	Adam, $\alpha = 5 \times 10^{-4}$, $\gamma = 0{,}99$
Tampon d'expérience	50 000 transitions, lots de 64
Début d'apprentissage	après 1 000 transitions collectées
Exploration	ϵ linéaire de 1,0 à 0,05 sur 30 000 pas
Durée d'un épisode	3 600 s simulées $\to$ 720 décisions
Nombre d'épisodes	150 (108 000 décisions)
Graines d'entraînement	$1000 + n$ pour l'épisode n (demande régénérée)
Graines d'évaluation	$\{7, 13, 21, 42, 77\}$, **jamais vues**

Demande de trafic. La demande OSM d'origine laissait le carrefour en écoulement libre (469 véh/h dispersés sur toute la ville, dont seulement 12 trajets traversant les branches étudiées) : il n'y avait rien à optimiser. Une demande dédiée a donc été construite, concentrée sur les quatre branches, avec un axe majeur chargé deux fois plus que l'axe secondaire et un profil temporel en trois paliers (montée, pointe, décrue). Cette asymétrie est ce qui rend le problème de régulation non trivial.

Régénération à chaque épisode. La demande est régénérée avec une graine différente à chaque épisode. L'agent ne peut donc pas mémoriser une séquence d'arrivées particulière ; il doit apprendre une politique fonction de l'état observé.

3 Résultats d'entraînement

Le Tableau 2 présente l'évolution des indicateurs internes au fil de l'entraînement.

TABLE 2 – Évolution des indicateurs d'entraînement (moyennes par tranche de 25 épisodes)

Épisodes	**1–25**	**26–50**	**51–75**	**76–100**	**101–125**	**126–150**
Attente moyenne (s)	413,3	91,0	50,5	43,2	38,9	**34,1**
File moyenne ($véh$)	8,35	4,31	3,13	2,85	2,65	**2,48**
Vitesse sur voies (km/h)	33,6	40,7	45,5	47,6	48,6	**49,4**

TABLE 3 – Valeurs aux jalons de l'entraînement

Métrique	**Épisode 1**	**Épisode 50**	**Épisode 100**	**Épisode 150**
Attente moyenne (s)	326,4	51,5	41,5	**27,0**
File moyenne ($véh$)	8,45	3,34	2,70	**2,16**
Perte de Huber	0,0	0,0859	0,0158	**0,0077**
Exploration ϵ	0,9772	0,0500	0,0500	0,0500
Changements de phase	213	216	221	218

Avertissement sur la métrique de vitesse. La « vitesse sur voies » est la moyenne de `getLastStepMeanSpeed` sur les voies contrôlées. Sur une voie *vide*, SUMO renvoie la vitesse autorisée. Cet indicateur mélange donc la vitesse réelle du trafic et la proportion de voies dégagées, ce qui explique qu'il approche 51 km/h alors que la limite urbaine est de 50 km/h. Il est utilisable comme signal de progression relative, mais **ne doit pas être présenté comme la vitesse des véhicules**. La vitesse réelle, calculée par véhicule à partir de `tripinfo.xml`, figure au Tableau 4 et vaut 8,16 m/s, soit 29,4 km/h.

3.1 Progression de l'attente moyenne

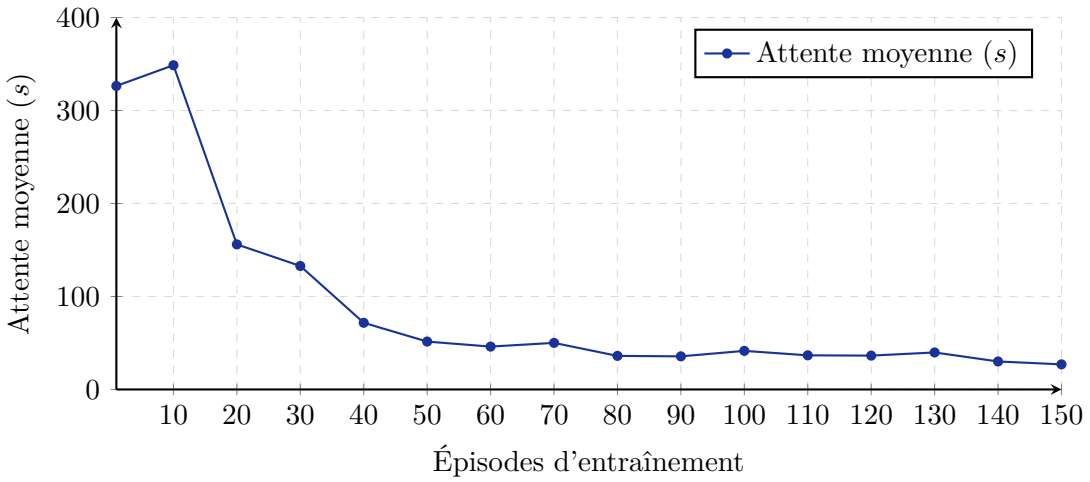


FIGURE 1 – Effondrement de l'attente moyenne, avec plateau atteint vers l'épisode 70.

3.2 Décroissance de la perte

La perte optimisée est la **perte de Huber** (`smooth_l1_loss`), moins sensible aux valeurs aberrantes que l'erreur quadratique — propriété utile ici, les récompenses étant occasionnellement extrêmes lors des blocages.



FIGURE 2 – Décroissance de la perte du réseau. La valeur nulle à l'épisode 1 traduit l'absence de mise à jour tant que le tampon n'a pas atteint 1 000 transitions.

3.3 Distribution des attentes moyennes

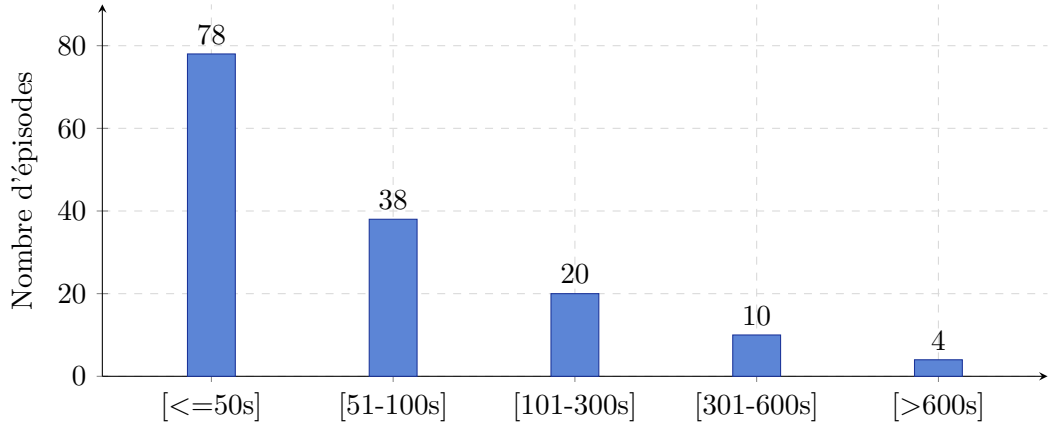


FIGURE 3 – Répartition des attentes moyennes sur les 150 épisodes. Les 14 épisodes au-delà de 300 s appartiennent tous à la phase d'exploration initiale.

4 Évaluation contre baselines

4.1 Pourquoi la progression interne ne prouve rien

Comparer l'agent entraîné à l'agent de l'épisode 1 revient à le comparer à un réseau de neurones *aléatoire*. Tout agent qui apprend quoi que ce soit bat son initialisation ; un tel écart ne renseigne donc pas sur l'utilité pratique du système. La seule question qui engage le projet est : **le régulateur appris fait-il mieux que le feu réellement en place ?**

Deux contrôleurs de référence sont donc évalués :

— **fixe** — le programme statique d'origine du réseau (27/6/6/6/27/6/6/6), c'est-à-dire le feu tel qu'il fonctionne aujourd'hui ;

— **actué** — un feu à détection de véhicules, obtenu par `netconvert -tls.default-type actuated -tls.rebuild` ($minDur = 8$ s, $maxDur = 50$ s). Le vert est prolongé tant que des véhicules se présentent. C'est la baseline exigeante.

Le type actionné du programme est vérifié à l'exécution : si le feu ne comporte aucun programme de type `ACTUATED`, l'évaluation s'interrompt par une erreur. Sans ce garde-fou, une baseline mal générée serait silencieusement identique au contrôleur fixe, et le résultat publié serait faux.

4.2 Résultats

Les trois contrôleurs sont évalués sur les cinq graines réservées, avec la même demande, sur des épisodes de 3 600 s. La politique apprise est exploitée sans exploration ($\epsilon = 0$). Les métriques proviennent de `tripinfo.xml`, donc mesurées *par véhicule*.

TABLE 4 – Comparaison des trois contrôleurs (5 graines inédites, écart-type inter-graines)

Contrôleur	**Temps perdu** (s)	**Attente** (s)	**Trajet** (s)	**Vitesse** (m/s)	Δ
fixe	$31{,}5 \pm 0{,}5$	$23{,}8 \pm 0{,}4$	48,5	6,37	référence
actué	$34{,}2 \pm 1{,}1$	$25{,}8 \pm 0{,}8$	51,2	5,72	+8,7 %
DQN	$\mathbf{15{,}5 \pm 0{,}6}$	$\mathbf{9{,}0 \pm 0{,}5}$	**32,7**	**8,16**	**−50,7 %**

Le régulateur appris divise le temps perdu par deux et l'attente par 2,6 face au feu en place.

Entre 843 et 907 véhicules terminent leur trajet selon la graine, avec moins de 1 % d'écart entre contrôleurs : les trois écoulent bien la même demande, et la comparaison porte donc sur la qualité de la régulation, non sur un débit différent.

4.3 Le gain moyen cache-t-il des véhicules sacrifiés ?

Une moyenne peut s'améliorer en dégradant une minorité : un contrôleur qui fluidifie l'axe majeur au prix de quelques véhicules bloqués très longtemps sur l'axe secondaire afficherait un bon score moyen tout en étant inacceptable. La question se vérifie sur la distribution complète, et non sur la moyenne seule (4 386 véhicules pour l'agent, 4 360 pour le contrôleur fixe, cumulés sur les cinq graines).

TABLE 5 – Distribution du temps d'attente par véhicule (quantiles, en secondes)

Contrôleur	n	**moyenne**	**médiane**	p_{90}	p_{99}	**maximum**
fixe	4 360	23,8	20	55	89	127
actué	4 375	25,8	24	53	76	136
DQN	4 386	**9,0**	**5**	**25**	**45**	**89**

L'agent domine à *tous* les quantiles, y compris au maximum : le véhicule le plus pénalisé sous sa régulation attend 89 s, contre 127 s sous le feu fixe et 136 s sous le feu actionné. Le gain n'est donc pas obtenu par report sur une minorité — le pire cas s'améliore de 30 % en même temps que la moyenne. C'est une propriété qui ne se déduisait pas du Tableau 4 et qui devait être vérifiée séparément.

4.4 Un résultat contre-intuitif : la baseline actionnée est moins bonne que le feu fixe

Le feu actionné dégrade les performances de 8,7 % par rapport au feu fixe. Ce résultat surprend, la détection de véhicules étant réputée supérieure à un programme statique. Il s'explique par la structure de la demande.

La logique actionnée prolonge le vert d'une branche tant que des véhicules s'y présentent, sur un critère purement *local*. Or l'axe majeur du carrefour est chargé deux fois plus que l'axe secondaire : le vert de l'axe majeur est donc systématiquement étendu jusqu'à $maxDur$, et l'axe secondaire est privé de vert. Le programme fixe, lui, garantit un tour de rôle : moins réactif, mais équitable.

Cette faiblesse est précisément ce que le DQN exploite. Sa décision porte sur les files observées *sur les quatre branches simultanément*, ce qui lui permet d'arbitrer entre elles plutôt que de réagir à une seule.

Ce résultat est conservé tel quel plutôt que d'être écarté : une baseline qui échoue pour une raison identifiée est une information, et elle délimite le domaine de validité de la conclusion — le gain observé tient à l'asymétrie de la demande.

5 Analyse critique et limites

5.1 Un défaut de sélection du modèle, détecté et corrigé

La sauvegarde du « meilleur » modèle était initialement pilotée par le retour cumulé de l'épisode. Ce critère est vide de sens dans cette formulation : la récompense étant une différence

$(W_{t-1} - W_t)/100$, sa somme sur un épisode se télescope en

$$\sum_{t=1}^{T} \frac{W_{t-1} - W_t}{100} = \frac{W_0 - W_T}{100},$$

soit une quantité qui ne dépend que des états initial et final — du bruit autour de zéro, ce que confirme la colonne `reward` du journal.

Conséquence mesurée : le modèle ainsi retenu (aux alentours de l'épisode 50) atteint $-32{,}3\,\%$ de temps perdu, contre $-50{,}7\,\%$ pour le modèle de l'épisode 150. Le critère a été remplacé par l'attente moyenne de l'épisode, évaluée uniquement une fois ϵ au plancher (sinon l'exploration résiduelle fausse la comparaison).

Ce défaut est rapporté parce qu'il illustre un risque général : une métrique de sélection non vérifiée peut faire publier un modèle sensiblement inférieur à celui dont on dispose, sans qu'aucun signal d'erreur n'apparaisse.

5.2 L'agent a-t-il simplement ré-appris un cycle fixe ?

Le nombre de changements par épisode se concentre dans $[188, 228]$, de moyenne $\mu = 215{,}6$ — une régularité qui suggère un cycle. L'argument par l'absurde (un cycle fixe ne peut pas diviser par deux le temps perdu d'un cycle fixe) est recevable mais indirect. La question est donc tranchée par la mesure directe (`rl/adaptivity.py`, cinq graines d'évaluation, 1 083 verts observés pour l'agent contre 795 pour le contrôleur fixe).

Un test invalide, à écarter explicitement. L'idée naturelle est de corréler la durée d'un vert à la file de la branche desservie. Appliquée au contrôleur *fixe*, cette corrélation vaut $r = +0{,}54$: un feu strictement statique paraît donc « réagir » au trafic. L'explication est un confondant de topologie — les phases de l'axe majeur ont à la fois plus de vert (33 s contre 12 s) et plus de file. La corrélation mesure la structure du cycle, pas une réaction à l'état. **Elle ne peut servir de preuve dans aucun sens** et est écartée.

Test valide : la variance intra-phase. Un cycle fixe a, par construction, un écart-type *rigoureusement nul* sur la durée d'une même phase. C'est sa signature, et elle est falsifiable.

Table 6 – Modulation de la durée du vert, à phase constante (5 graines)

	fixe	**actué**	**DQN**
Écart-type intra-phase	**0,00** s	1,38 s	**3,53** s
Plage des durées observées	12–33 s	14–44 s	**15–55** s
Choix de la phase la plus chargée	—	—	**79,2** %

Le contrôleur fixe donne 0,00 s, comme attendu. L'agent module la durée d'une même phase de 15 à 55 s selon l'état observé : **l'hypothèse du cycle fixe est réfutée par la mesure**, et non seulement par l'absurde. Par ailleurs, lorsque plusieurs actions sont autorisées, l'agent sélectionne la phase dont la file est la plus longue dans 79,2 % des cas (1 208 décisions), contre 25 % pour un choix uniforme sur quatre phases.

La stabilité du *nombre* de changements s'explique enfin par les contraintes de sécurité : avec un vert minimal de 10 s et maximal de 60 s sur 3 600 s, ce nombre est encadré par construction dans $[60, 360]$. La marge de manœuvre porte sur le choix de la phase et l'instant du basculement, non sur la fréquence globale.

Effet de plancher. La file à l'ouverture du vert vaut en moyenne 0,31 véhicule chez l'agent, contre 2,24 chez le contrôleur fixe. L'agent maintient le carrefour si dégagé qu'il ne reste presque plus de variance à expliquer : c'est un succès du contrôleur, mais cela rend tout test fondé sur cette variable peu informatif — raison supplémentaire de s'appuyer sur la variance intra-phase.

5.3 Une phase abandonnée : optimisation réelle, et son coût

La mesure a révélé un comportement non anticipé : **l'agent n'ouvre jamais la phase 1** (0 vert sur 1 083 observés ; 410, 451 et 222 pour les phases 0, 2 et 3). L'analyse de la topologie du feu montre qu'il ne s'agit pas d'une défaillance.

Table 7 – Mouvements desservis par chaque phase verte

Phase	Liens verts	Mouvements
0	10	axe majeur tout droit (`G`) *et* tourne-à-gauche en permissif (`g`)
1	**4**	**tourne-à-gauche de l'axe majeur uniquement, en protégé (`G`)**
2	6	axe secondaire tout droit
3	4	tourne-à-gauche de l'axe secondaire, en protégé

La phase 1 est un **sous-ensemble strict de la phase 0** : tous les mouvements qu'elle autorise le sont déjà en phase 0, en permissif au lieu de protégé. L'agent a donc identifié qu'elle n'apporte aucune capacité supplémentaire et redistribue ses ≈ 12 s de vert aux mouvements qui en tirent parti. C'est une optimisation de fond, que ni le contrôleur fixe ni le contrôleur actionné ne peuvent découvrir, l'un comme l'autre parcourant les phases dans l'ordre programmé.

Le coût, qui doit être énoncé. Faire passer un tourne-à-gauche du protégé au permissif transfère la résolution du conflit au conducteur, qui doit trouver un créneau dans le flux opposé. En simulation, le coût reste marginal (2 freinages d'urgence sur cinq épisodes). Sur le terrain, ce compromis relève d'une décision d'ingénierie de sécurité et **non de l'agent** : la fonction de récompense ne mesure que le temps d'attente, et rien n'y représente le risque d'accident. Un agent optimise ce qu'on lui donne à optimiser — ici, il a trouvé un gain que le cahier des charges implicite n'interdisait pas.

La correction appropriée n'est pas de brider l'agent après coup mais d'inscrire la contrainte dans le problème : garantir un vert protégé périodique par mouvement, ou pénaliser explicitement les conflits permissifs dans la récompense.

5.4 Métrique de vitesse trompeuse sur les voies

Comme signalé en Section 3, la vitesse moyenne par voie intègre les voies vides, où SUMO renvoie la vitesse autorisée. Un carrefour parfaitement dégagé maximise donc cette métrique, qui mesure en partie l'absence de trafic. Les conclusions du présent rapport reposent pour cette raison sur `tripinfo.xml`, agrégé par véhicule.

5.5 Équité par mouvement non garantie

La récompense n'agrège que le temps d'attente *total* : rien n'y interdit de privilégier durablement un mouvement au détriment d'un autre. Le Tableau 5 montre qu'aucun véhicule n'est sacrifié en pratique — le pire cas s'améliore lui aussi — mais c'est une propriété *constatée*, non une propriété *garantie* par la formulation. L'abandon de la phase 1 (Section 5.3) en est

l'illustration : l'agent a supprimé un vert protégé parce qu'aucun terme de la récompense ne le protégeait.

Une formulation destinée au déploiement devrait inscrire cette contrainte dans le problème plutôt que d'espérer qu'elle émerge : plafond d'attente par mouvement, garantie de service périodique, ou pénalité sur la variance inter-branches.

5.6 Portée limitée à un carrefour isolé

L'agent régule une intersection unique. Rien ne garantit que la politique apprise reste bénéfique en réseau : fluidifier un carrefour peut déplacer la congestion vers le suivant. La validation d'un déploiement réel exigerait une évaluation sur un corridor, avec une coordination inter-carrefours.

5.7 Absence de répétitions indépendantes

Les résultats proviennent d'*une* exécution d'entraînement évaluée sur cinq graines. L'écart-type reporté mesure la variabilité *du trafic*, non celle de *l'apprentissage*. Trois à cinq entraînements avec des graines d'initialisation distinctes seraient nécessaires pour établir la stabilité de la méthode elle-même. L'ampleur de l'écart observé ($-50{,}7\,\%$ contre $\pm 0{,}6$ s d'écart-type) rend toutefois improbable qu'il s'agisse d'un artefact.

5.8 Écart entre simulation et terrain

SUMO modélise un trafic homogène de véhicules particuliers. Le trafic réel de Thiès comporte deux-roues, charrettes et piétons traversant hors passage, dont les comportements ne sont pas représentés. Les gains rapportés valent dans le modèle ; ils constituent une borne optimiste pour le terrain.

6 Conclusion et perspectives

6.1 Rappels théoriques

$$\textbf{Équation de Bellman optimale} \quad Q^*(s,a) = \mathbb{E}_{s'}\left[r + \gamma \max_{a'} Q^*(s',a') \mid s,a\right] \tag{1}$$

$$\textbf{Cible du Double DQN} \quad y_t = r_t + \gamma\, Q_{\bar{\theta}}\big(s_{t+1},\ \arg\max_{a'} Q_{\theta}(s_{t+1},a')\big) \tag{2}$$

Le découplage entre le réseau qui *choisit* l'action (θ) et celui qui l'*évalue* ($\bar{\theta}$) atténue la surestimation systématique des valeurs Q propre au DQN classique.

6.2 Résultats établis

1. Le régulateur appris réduit le temps perdu par véhicule de **50**,**7** % face au feu fixe en place (31,5 s $\rightarrow$ 15,5 s), sur cinq graines de trafic jamais rencontrées à l'entraînement.
2. L'attente moyenne par véhicule diminue de **62**,**2** % (23,8 s $\rightarrow$ 9,0 s).
3. Le gain n'est pas obtenu au détriment d'une minorité : l'agent domine à *tous* les quantiles, le véhicule le plus pénalisé attendant 89 s contre 127 s sous le feu fixe.
4. Le caractère adaptatif est établi par la mesure et non par l'argument : la durée d'une même phase varie de 15 à 55 s (écart-type intra-phase de 3,53 s, contre 0,00 s pour le feu fixe), et l'agent sert la branche la plus chargée dans 79,2 % des arbitrages.
5. Le contrôle effectif du feu par l'agent est vérifié par un test dédié, et non supposé.
6. La baseline actionnée s'avère inférieure au feu fixe sur cette demande asymétrique, ce qui délimite le domaine de validité du résultat.

7. L'agent a découvert que la phase 1 est redondante en capacité et l'a supprimée — gain réel, mais au prix du passage de tourne-à-gauche protégés en permissifs, arbitrage qui relève de l'ingénieur et non de l'agent.

6.3 Travaux futurs

— Répéter l'entraînement avec plusieurs graines d'initialisation pour établir la stabilité de la méthode, et non seulement celle d'un modèle.
— Réintroduire l'équité par mouvement dans la formulation (vert protégé périodique garanti, ou pénalité sur les conflits permissifs) et vérifier le coût en performance de cette contrainte.
— Étendre à un corridor de plusieurs carrefours coordonnés, pour vérifier que le gain local ne se traduit pas par un report de congestion.
— Éprouver la robustesse sur des scénarios dégradés : pic de demande, panne de détecteur, arrivée de véhicule prioritaire.
— Publier poids (`dqn_best.pt`), journal d'entraînement et scripts d'évaluation afin de permettre la reproduction indépendante des résultats.

6.4 Reproductibilité

```
py -3.14 -m rl.smoke_test                          # vérifie le contrôle du feu
py -3.14 -m rl.train --episodes 150                # entraînement (~45 min)
py -3.14 -m rl.evaluate --model outputs/dqn_best.pt  # tableau comparatif
py -3.14 -m rl.adaptivity --seeds 7 13 21 42 77    # preuve d'adaptativité
py -3.14 -m rl.demo --controller dqn --seed 7      # démonstration visuelle
```

Avertissement de reproductibilité. Les fichiers `tripinfo_*.xml` du répertoire `outputs/` sont écrasés à chaque exécution, et `rl.train` réécrit `training_log.csv` depuis zéro. Un artefact présent sur le disque ne correspond donc pas nécessairement au modèle courant : lors de la rédaction, des `tripinfo` antérieurs au remplacement de `dqn_best.pt` donnaient $-32{,}3\,\%$ au lieu de $-50{,}7\,\%$. Toute vérification doit relancer `rl.evaluate` plutôt que relire les fichiers existants.